

Разработка real-time интерфейса с поддержкой горизонтального масштабирования WebSocket-соединений для системы отслеживания задач

Защита выпускной квалификационной работы

Студент: Зайцев А. С., группа 41313

Руководитель: канд. техн. наук, доцент А. В. Фомин

Санкт-Петербург, 2026

Проблема и цель

Single-node SignalR ограничивает горизонтальное масштабирование

Проблема

Сведения о подключениях и группах SignalR хранятся в памяти процесса.

При нескольких экземплярах backend событие, созданное на одном узле, не доходит до клиента, подключенного к другому узлу.

Real-time пространство фрагментируется на независимые острова.

Цель работы

Real-time интерфейс системы отслеживания задач, в котором WebSocket-соединения обслуживаются несколькими экземплярами app-арі, а события об изменении сущностей доставляются клиентам независимо от узла подключения.

Современный контекст

Интерфейсы становятся событийными; AI-инструменты усиливают ожидание живой синхронизации

Рост сложности продуктов

Пользователь работает не с одной формой, а с отчётом, багами, комментариями, вложениями, статусами и участниками.

Изменения должны становиться видимыми без ручного обновления страницы.

AI-ready сценарии

Фоновый сервис или ассистент может предложить исполнителя, дополнить описание, создать комментарий или изменить статус.

В ВКР это контекст актуальности, а не заявление о готовой AI-функции.

REST + real-time

HTTP-запрос фиксирует изменение состояния.

SignalR-событие распространяет изменение активным участникам, включая клиентов на других backend-узлах.

Предметная область

Real-time связан с сущностями продукта, а не с абстрактным WebSocket-демо

**Система отслеживания задач: отчёты · баги · комментарии · вложения · ссылки · шаги
воспроизведения · статусы**

Открытие отчёта

JoinReportGroupAsync

Клиент вступает в группу отчёта и получает только релевантные события.

Изменение сущностей

**ReceiveReportPatch
ReceiveBugCreate
ReceiveCommentUpdate**

События применяются к состоянию интерфейса.

Восстановление

**onreconnected
JoinReportGroupAsync**

После reconnect клиент повторно восстанавливает подписку на текущий отчёт.

Анализ подходов к масштабированию

Шесть рассмотренных вариантов; для реализации выбран подход 4

Один экземпляр backend
Нет масштабирования; один узел — единственная точка отказа

Несколько экземпляров без общего канала
События не доходят до клиентов, подключённых к другим узлам

Привязка клиента к узлу (sticky)
Клиент остаётся на одном узле, но события на другие не приходят

Внешняя шина событий
Доставка между узлами есть, но требуется отдельная схема событий

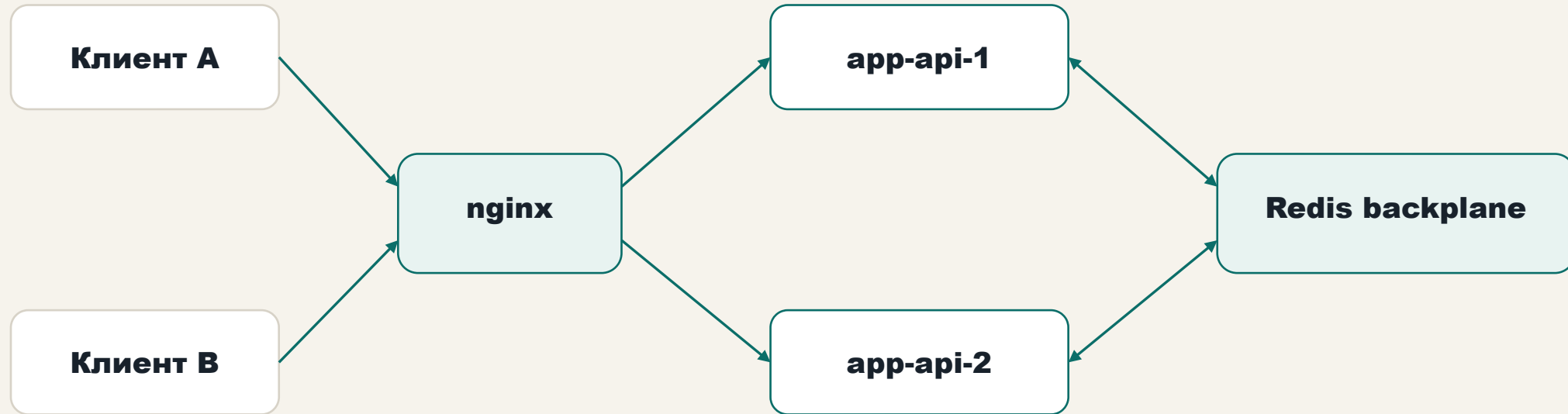
Облачный real-time сервис
Готовое решение, но снижает контроль над собственным контуром

**Redis backplane для SignalR —
выбрано**
**Штатный механизм, без
перестройки кода, подходит для
событий интерфейса**

Критерии выбора: запускается на локальном стенде, сохраняет существующий код доставки событий, штатно поддерживается в ASP.NET Core SignalR, подходит для коротких событий интерфейса и легко воспроизводится.

Целевая архитектура

Два экземпляра app-api за nginx, общий Redis backplane для межузловой доставки



Событие публикуется в Redis · доставляется обоим узлам · группы SignalR остаются локальным runtime-состоянием

Оба экземпляра подключены к общей PostgreSQL; на схеме показан только канал real-time-доставки.

Программная реализация

Backend, frontend, тесты и проверочный клиент закрывают задачу как разработку ПО

Backend

AddStackExchangeRedis через
REDIS_CONNECTION_STRING
GetConnectionDiagnosticsAsync
SendToGroupAsync с eventId и GroupExcept

Frontend

WebSocket transport
automatic reconnect
повторный JoinReportGroupAsync после reconnect

Тесты

reconnectJoinHandler проверяет отсутствие лишнего join, повторное вступление и использование актуального reportId.

Проверочный клиент

delivery · rejoin · failover · series
Фиксирует serverInstanceId, connectionId, событие и задержку.

Программа испытаний

Два режима по одному коду · четыре сценария проверки

Два режима

- Без межузлового канала — воспроизведение проблемы
- С межузловым каналом — проверка решения

Один и тот же код, один и тот же сценарий;
отличие — наличие Redis backplane.

Четыре сценария

- Одиночная доставка между узлами
- Серия из 30 итераций
- Восстановление подписки после разрыва
- Переключение на другой узел после отказа

Главный результат

Один код и один сценарий — разница только в наличии Redis-канала

Без backplane

**ReceiveReportPatch
timed out · 10 000 мс**

**Событие, созданное на app-api-1,
не дошло до клиента на app-api-2**

С backplane

**ok: true · ReceiveReportPatch
получен
deliveryLatencyMs ≈ 9,5 мс**

**Клиент на app-api-2 получил
событие с app-api-1**

Сопоставление двух режимов на одном стенде показывает причинную связь между включением межузлового канала SignalR и успешной доставкой события.

Дополнительные проверки

Повторяемость, восстановление подписки и переключение после отказа узла

Серия из 30 итераций

Успешно: 30 из 30

min 3,8 мс

avg 7,2 мс

p50 6,1 мс

p95 13,8 мс

Подтверждена повторяемость доставки.

Восстановление подписки

После разрыва клиент создаёт новое соединение и снова вступает в группу отчёта.

переподключение 6,5 мс

доставка 15,2 мс

Подтверждена повторная подписка.

Переключение после отказа узла

Один backend остановлен; nginx переключает клиента на другой экземпляр.

переподключение 352,0 мс

доставка 5,4 мс

Подтверждено восстановление после отказа.

Ограничения

Решение предназначено для интерфейсной синхронизации, а не для жёсткого real-time

Ненулевая задержка

Событие проходит через app-api, Redis backplane и другой app-api.

В локальной серии: avg 7,2 мс, p95 13,8 мс.

Transient UI-events

Источник истины — PostgreSQL.

При сомнениях клиент восстанавливает согласованность повторным HTTP-запросом состояния.

Дальнейшее развитие

Durable broker/event log
версии сущностей
optimistic concurrency
outbox/inbox
повторная синхронизация
по версии

Вывод

Ограничение single-node WebSocket-архитектуры устранено за счёт распределённого real-time контура; тезис работы подтверждён экспериментально.

Дополнительный эффект — повышение отказоустойчивости: при отказе одного узла клиенты продолжают получать события через другой экземпляр backend, тогда как single-node архитектура такой возможности не предоставляет.

Проблема

Single-node SignalR: события не пересекают границу узла.

Решение

Несколько экземпляров backend за nginx + Redis backplane; общий код, переключение режима — переменной окружения.

Доказательство

**Доставка (без / с) · серия ·
восстановление подписки ·
переключение после отказа.**